

MUSTAFACODE

Lecture 16

Operator Overloading in C++ (Detailed Notes)
Object-Oriented Programming (CS304)

Course Code: **CS304**

Publisher: **mustafacode**

Compiled On: **May 29, 2026**

Lecture 16: Operator Overloading in C++ (Detailed Notes)

Operator Overloading in C++ (Detailed Notes)

◆ Introduction to Operator Overloading

C++ hume allow karta hai ke hum operators (`+` , `-` , `*` , `==` etc.) ko apni custom classes ke objects ke liye bhi use kar saken.

Is concept ko kehte hain:

Operator Overloading

◆ Real Life Example

Socho aap ke paas ek `Complex` number class hai.

Complex Number:

```
2 + 3i  
4 + 5i
```

Agar hum normally add karna chahen:

```
Complex c3 = c1.Add(c2);
```

Yeh thora lengthy aur less readable hai.

Operator overloading ke baad hum simply likh sakte hain:

```
Complex c3 = c1 + c2;
```

Bilkul normal mathematics ki tarah ✓

◆ Why Operator Overloading?

Without operator overloading:

```
c1.Add(c2);
```

With operator overloading:

```
c1 + c2;
```

Benefits:

- Code readable hota hai
- Easy to write
- Easy to maintain
- Mathematical expressions natural lagti hain

◆ Operators Built-in Types Ke Liye Already Overloaded Hote Hain

Example:

```
int a = 5 + 3;  
float b = 2.5 + 1.5;
```

Internally compiler functions call karta hai.

Example:

```
Add(int,int)  
Add(float,float)
```

◆ Operator Functions

Operator functions usually direct call nahi hoti.

Compiler automatically call karta hai.

Example:

```
c1 + c2
```

Internally:

```
c1.operator+(c2)
```

◆ Operators That Can Be Overloaded

```
+  
-  
*  
/  
%  
==  
!=  
<  
>  
<=  
>=  
++  
--  
[]  
()  
<<  
>>  
=  
+=  
-=  
*=  
/=  
new  
delete
```

◆ Operators That Cannot Be Overloaded

```
.  
.*  
::  
?:  
#  
##
```

◆ Why Some Operators Cannot Be Overloaded?

Kyun ke yeh operators actual object/function names ke sath directly kaam karte hain.

Example:

```
student.getRollNo();
```

Yahan `.` actual member function ko access kar raha hai.

◆ Precedence of Operators

Precedence decide karti hai konsa operator pehle chalega.

Operator overloading precedence ko change nahi karti.

Example:

```
c1 * c2 + c3
```

Pehle multiplication hogi.

◆ Associativity

Associativity bhi change nahi hoti.

Left Associative

```
c1 + c2 + c3
```

Means:

```
(c1 + c2) + c3
```

Right Associative

Assignment operator:

```
a = b = c;
```

Means:

```
a = (b = c);
```

◆ Important Rules

✓ Rule 1

Operator ke andar wahi logic likho jo operator represent karta hai.

Wrong Example:

```
+ operator ke andar subtraction
```

Yeh confusion create karega ❌

✓ Rule 2

Naya operator create nahi kar sakte.

Wrong:

\$

Syntax error ❌

◆ Arity of Operators

Arity ka matlab:

👉 operator kitne operands leta hai

Example:

```
a + b
```

+ binary operator hai.

Operator overloading arity change nahi karti.

◆ Unary Operators

Unary operators sirf ek operand lete hain.

Examples:

```
++  
--  
!
```

◆ Binary Operators

Binary operators do operands lete hain.

Examples:

```
+  
-  
*  
/  
==  
<  
>
```

◆ General Syntax of Operator Overloading



Member Function Syntax

```
return_type class_name::operator operator_symbol(parameters){  
    // code  
}
```



Non-Member Function Syntax

```
return_type operator operator_symbol(parameters){  
    // code  
}
```

◆ Binary Operator Overloading Syntax

Member Function

```
TYPE class_name::operator operator_symbol(TYPE rhs){  
    // code  
}
```

◆ Important Point

Kam az kam ek parameter user-defined type ka hona chahiye.

Wrong Example:

```
int operator+(int,int);
```

Error ❌

◆ Example: Overloading + Operator



Complete Program

```
#include<iostream>
using namespace std;

class Complex{

private:
    double real;
    double img;

public:

    // Constructor
    Complex(double r=0,double i=0){
        real = r;
        img = i;
    }

    // Operator OverLoading
    Complex operator +(const Complex & rhs){

        Complex temp;

        temp.real = real + rhs.real;
        temp.img = img + rhs.img;

        return temp;
    }

    // Display Function
    void display(){

        cout << real << " + " << img << "i" << endl;
    }
};

int main(){

    Complex c1(2,3);
    Complex c2(4,5);
    Complex c3;
```

```
c3 = c1 + c2;  
  
c3.display();  
  
return 0;  
}
```

◆ Output

```
6 + 8i
```

◆ Internally Compiler Kya Karta Hai?

```
c1 + c2
```

Compiler convert karta hai:

```
c1.operator+(c2)
```

◆ Cascaded Expressions

Operator overloading ka sabse bada benefit:

```
c1 + c2 + c3
```

Compiler internally:

```
(c1.operator+(c2)).operator+(c3)
```

◆ Why Return Type Should Be Object?

Agar return type object ho:

```
Complex operator+(...)
```

To hum likh sakte hain:

```
c1 + c2 + c3
```

◆ What Happens If Return Type is void?

Example:

```
void operator+(...)
```

Problems:

- Chaining possible nahi
- Assignment possible nahi
- Existing object modify hota hai
- Code less readable hota hai

◆ Example of void Return Type

```
class Complex{  
    public:  
  
    void operator +(const Complex & rhs){  
  
        real = real + rhs.real;  
        img = img + rhs.img;  
    }  
};
```

Usage:

```
c1 + c2;  
c1 + c3;
```

Final result `c1` mein store hoga.

✗ Drawbacks of void Return Type

1. Assignment Possible Nahi

Wrong:

```
Complex t = c1 + c2;
```

2. Cascaded Expressions Work Nahi Karengi

Wrong:

```
c1 + c2 + c3
```

3. Existing Object Modify Hota Hai

Result nayi object mein nahi aata.

4. Code Less Readable

5. Debugging Difficult

6. Maintenance Hard

◆ Real Life Analogy

Socho calculator hai.

Without operator overloading:

```
Add(number1,number2)
```

With operator overloading:

```
number1 + number2
```

Second approach zyada natural aur easy lagti hai 

◆ Summary

Concept	Explanation
Operator Overloading	Operators ko custom classes ke liye redefine karna
Benefit	Readable & maintainable code
Unary Operator	One operand
Binary Operator	Two operands
Precedence	Change nahi hoti
Associativity	Change nahi hoti
New Operator	Create nahi kar sakte
Best Practice	Object return type use karo

◆ Final Important Point

Operator overloading ka purpose:

-  Code ko natural banana
-  Readability improve karna
-  Complex mathematical expressions ko easy banana
-  Maintenance easy banana